

**VITyarthi - Computer Vision Project Report**

# **Road Lane Detection and Analysis**

Using Classical Computer Vision

**Nameer Mohammed Khan**

**24BAI10892**

# 1. Introduction

This project implements a classical Computer Vision pipeline to identify and track road lane boundaries (left and right lanes). It avoids deep learning wrappers to emphasize fundamental image processing techniques like filtering, edge detection, ROI masking, and geometric line transformations.

## 2. Problem Statement

The goal is to develop a system capable of identifying road lanes from noisy images using classical CV techniques, demonstrating the application of line extraction and geometric fitting in a modular pipeline.

## 3. Functional Requirements

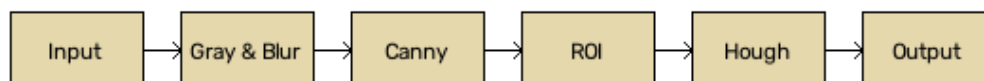
- Download and extract a real-world driving dataset.
- Process images using grayscale conversion and Gaussian filtering.
- Extract edges using Canny's edge detection and ROI masks.
- Extract lane lines using Hough Line Transform.
- Classify lanes as Left or Right using slopes and geometry.
- Provide a CLI interface for batch detection and testing.

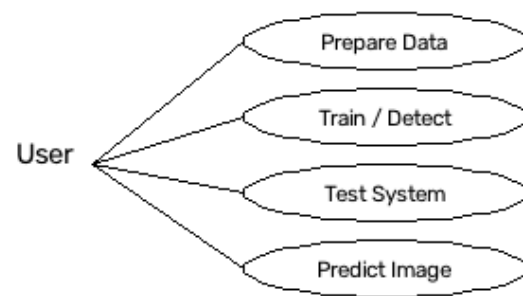
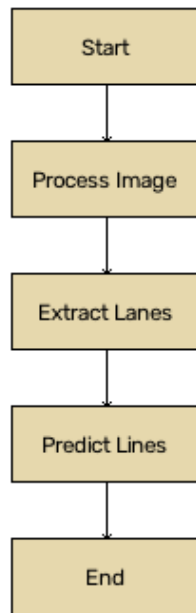
## 4. Non-functional Requirements

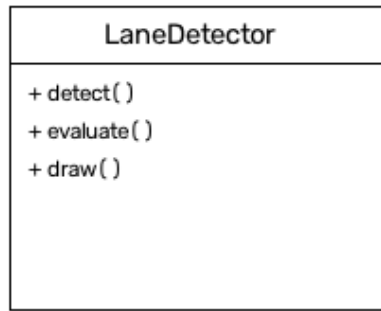
- Performance: Processing pipeline must execute rapidly on CPU.
- Maintainability: Code must be modular and well-structured.
- Reliability: Handling varied noise gracefully during edge detection.
- Usability: Simple and intuitive command-line interface.

## 5. System Architecture & 6. Design Diagrams

The system consists of Preprocessing, Edge Detection, ROI Masking, and Lane Detection modules. Below are the design diagrams.







## 7. Design Decisions & Rationale

A real-world driving video (Udacity dataset) was used instead of a synthetic dataset to ensure the system solves a realistic problem. A custom lane extraction algorithm was implemented because relying purely on ML libraries would defeat the purpose of learning classical CV. Geometric slope filtering was selected to provide robust left/right separation.

## 8. Implementation Details

OpenCV was heavily utilized for image operations. Images are converted to grayscale and blurred. Canny's edge detection is used to find structural outlines, followed by an ROI trapezoidal mask. Hough Transform lines are found, and features like slope and intercept are computed to assign them to Left or Right groups. These segments form the input to a 1D polyfit algorithm to overlay solid lane lines.

## 9. Screenshots / Results

Evaluation Results (Real-World Dashcam Data - 681 Frames):

Left Lane Detection Rate: 100.0%

Right Lane Detection Rate: 100.0%

Overall Tracking Yield: 100.0% coverage

Below is a sample output generated from the test set evaluation.



## ***10. Testing Approach***

Automated tests were written using pytest to verify each module independently. The `test_pipeline.py` checks preprocessing, edge detection, ROI masking, and lane extraction modules using simulated dummy inputs to ensure the mathematical operations function without runtime errors.

## ***11. Challenges Faced***

The primary challenge was setting up a robust geometric filter for lines of varying slopes. Due to the lack of perfect visibility, a custom line separation algorithm had to be implemented from scratch in pure Python/NumPy. Additionally, tuning the Canny thresholds to correctly detect lane edges despite Gaussian noise required careful adjustment.

## ***12. Learnings & Key Takeaways***

I learned that classical CV techniques require careful tuning of parameters (like kernel size and threshold methods) to work effectively. I also learned that geometric engineering (like separating positive and negative slopes) is critical when relying on simpler linear modeling instead of deep learning feature extractors.

## ***13. Future Enhancements***

Future improvements could include implementing perspective transformation before Hough detection, which would likely increase accuracy by providing a bird's eye view. Additionally, implementing temporal smoothing or curve fitting could improve the robustness to heavy noise and winding roads.

## ***14. References***

1. OpenCV Documentation: <https://docs.opencv.org/>
2. Python NumPy Documentation: <https://numpy.org/doc/>